

ROBOTICS · LESSON 4

The Rover

Two DC motors through an L298N H-bridge — the read → decide → drive loop now drives wheels. Forward, back, and turn.

ROBO-004 · ~45 MIN · INTERMEDIATE

1. The one idea

Locomotion. Two DC motors through an **L298N H-bridge** make the robot *move* — forward, back, and turn — under the same `read → decide → drive` loop, now driving wheels instead of a servo.

2. Why it matters

A servo holds an angle; a DC motor spins for locomotion. But you cannot drive a motor from a GPIO pin: motors pull far more current than a pin can source, and they kick back voltage spikes (back-EMF) that destroy it. The **H-bridge** is the standard answer — a driver that takes your low-power logic signals and switches high-power motor current, and lets you reverse direction by flipping which way the current flows. This is the gateway from "things that twitch" to "things that drive."

3. Parts & wiring

- ESP32 dev board
- 2× DC gear motors on a chassis (with wheels)
- L298N motor-driver module
- A motor battery pack (e.g. 6×AA or a 7.4V pack) — **separate from the ESP32's USB power**

ESP32 pin	L298N	Drives
GPIO25	IN1	left motor, direction A
GPIO26	IN2	left motor, direction B
GPIO27	ENA	left motor, speed (PWM)
GPIO14	IN3	right motor, direction A
GPIO4	IN4	right motor, direction B
GPIO13	ENB	right motor, speed (PWM)

Power: motor battery + to the L298N's VS (12V) terminal, battery – to L298N GND, and **tie that GND to the ESP32's GND**. The motors run from the battery through the driver; the ESP32 runs from USB. Share only ground.

*Two invisible rules. First: **never run motors from the ESP32** — they will brown it out and reset it mid-drive. Give them their own battery through the L298N. Second: the L298N drops about 2V, so a 7.4V pack gives the motors ~5.4V — size the battery for the speed you want.*

4. Predict (do this before uploading)

- With `IN1 = HIGH`, `IN2 = LOW`, which way does the left motor spin?
- To **turn right in place**, what should each wheel do?
- At PWM 200 of 255, roughly what fraction of full speed is that?

5. Build it — one drive() function

```
const int IN1 = 25, IN2 = 26, ENA = 27;    // left motor
const int IN3 = 14, IN4 = 4, ENB = 13;    // right motor

void setup() {
  int pins[] = { IN1, IN2, IN3, IN4, ENA, ENB };
  for (int p : pins) pinMode(p, OUTPUT);
}

// dir: +1 forward, -1 back, 0 stop; speed: 0-255
void drive(int leftDir, int rightDir, int speed) {
  digitalWrite(IN1, leftDir > 0); digitalWrite(IN2, leftDir < 0);
  digitalWrite(IN3, rightDir > 0); digitalWrite(IN4, rightDir < 0);
  analogWrite(ENA, speed);      analogWrite(ENB, speed);
}

void loop() {
  drive(+1, +1, 200); delay(1000);    // forward
  drive(0, 0, 0); delay(500);        // stop
  drive(+1, -1, 200); delay(600);    // spin right (left fwd, right back)
  drive(0, 0, 0); delay(500);
}
```

The whole locomotion API is one function: `drive(left, right, speed)`. Two wheels with independent direction and a shared speed give you forward, reverse, gentle turns, and spin-in-place — that is **differential drive**, and naming it is the lesson. Direction lives on the IN pins; speed is PWM on the enables.

6. Test against your prediction

Prop the chassis up so the wheels spin free, then upload. It should go forward, stop, and spin. Did the spin turn the way you predicted? If a wheel runs backward, swap that motor's two wires (or its IN pair) — don't fight it in code.

7. What goes wrong (pre-loaded debugging)

- **Motors don't move and the ESP32 keeps resetting** → you're pulling motor current through the board. Separate battery on the L298N VS, grounds tied. The number-one rover bug.
- **One wheel spins the wrong way** → that motor's leads are reversed. Swap the two motor wires or its IN pair.
- **It veers instead of going straight** → open-loop DC motors aren't matched. Trim one side's PWM — and note this is exactly why closed-loop control (encoders) exists.
- **Weak or no motion** → dead battery, the L298N's ~2V drop, or an enable pin left LOW (PWM 0 = motor disabled).
- **Resets the instant motors start** → inrush current sags the battery. Use a fresh pack, a common ground, and a bulk capacitor across the motor supply.
- **Board won't boot with the driver attached** → avoid the ESP32 strapping pins (GPIO0, 2, 5, 12, 15) for IN lines — a driver pulling one at boot can stop the chip from starting. The pins above dodge them on purpose.

8. Checkpoint

You have a rover. The same `read → decide → drive` loop now drives wheels: `drive(left, right, speed)` turns logic into locomotion through an H-bridge. It can go anywhere — it just doesn't know where it is, or what's in front of it, *yet*.

9. Push further

- Ramp the speed up instead of slamming to full — a slew-rate limit, the Lesson 2 `maxStep` idea applied to motors.
- Bolt on the Lesson 3 ultrasonic sensor as the rover's eyes → Lesson 5.

- Add wheel encoders to measure what the motors actually did — the door to closed-loop speed control.